

Département Mathématiques & Informatique

Cours structures de données

Chap5

Expressions lambda et Fonctions d'ordre supérieur

Expressions Lambda

- Comprendre la syntaxe et le fonctionnement des expressions lambda.
- Différencier les expressions lambda des fonctions classiques.
- Appliquer les lambdas à des tâches simples et courantes.

I. Expressions Lambda

Introduction

- Une **expression lambda** en Python est une **fonction anonyme**, utile lorsqu'une fonction simple doit être définie rapidement sans utiliser le mot-clé **def**.
- Elle est particulièrement utile dans les cas où une fonction est **temporaire** ou **utilisée en tant qu'argument pour une autre fonction**.
- Elle permet de **réduire le code** mais sont limitées à une seule expression.
- Elles peuvent être utilisées comme :
 - **Valeur par défaut d'un paramètre formel**
 - **Paramètre effectif**
 - **Exceptionnellement comme valeur d'une affectation**
- Elle évite de créer des fonctions à usage unique
- Utilisée principalement avec **map()**, **filter()**, **reduce()**, etc.

I. Expressions Lambda

Syntaxe

```
lambda arguments : expression
```

Où

- Arguments : Les paramètres de la fonction, similaires à ceux d'une fonction normale.
- Expression : Une unique expression calculée et renvoyée comme résultat.

Equivalence :

```
func = lambda arguments : expression
```

est équivalent à :

```
def func(arguments):  
    return expression
```

I. Expressions Lambda

Exemple 1 : additionner | multiplier deux nombres

```
add = lambda x, y : x + y  
mult= lambda x,y: x*y
```

```
# ou bien  
def add_func(x,y):  
    return x+y  
def add_func(x,y):  
    return x+y
```

```
>>> print("3+5=", add(3, 5), ", 3*5=", mult(3, 5))  
3+5=8 , 3*8=24
```

```
>>> print("3+5=", add_func(3, 5), ", 3*5=", mult_func(3, 5))  
3+5=8 , 3*8=24
```

I. Expressions Lambda

Exemple 2 : ordre de tri

```
def trier(l, en_ordre = lambda x, y: x <= y):  
    est_trie = False  
    while not est_trie:  
        est_trie = True  
        for i in range(len(l)-1):  
            if not en_ordre (l[i], l[i+1]):  
                l[i], l[i+1] = l[i+1], l[i]  
                est_trie = False
```

```
>>> l=[3,2,8,1,5,98,3]  
>>> trier(l)  
>>> l  
>>> [1, 2, 3, 3, 5, 8, 98]  
  
>>> trier(l,lambda x,y : x > y)  
>>> l  
>>> [98, 8, 5, 3, 3, 2, 1]
```

I. Expressions Lambda

Exemple 3 : généralisation du tri (1 / 2)

```
def trier(l,  
    en_ordre =lambda x,y: x <= y,  
    a_comparer=lambda x: x  
):  
    est_trie = False  
    while not est_trie :  
        est_trie = True  
        for i in range(len(l)-1):  
            if not en_ordre (a_comparer(l[i]),a_comparer(l[i+1])):  
                l[i], l[i+1] = l[i+1], l[i]  
                est_trie = False
```


I. Expressions Lambda

Exemple 3 : généralisation du tri (2 / 2)

```
>>> l=[complex(3,1), complex(1,3), complex(2,2)]  
>>> trier(l)
```

TypeError: '<=' not supported between instances of 'complex' and 'complex'

```
>>> trier(l, a_comparer=lambda z:z.real)  
>>> l  
[(1+3j), (2+2j), (3+1j)]
```

```
>>> trier(l, a_comparer=lambda z:z.imag)  
>>> l  
[(3+1j), (2+2j), (1+3j)]
```

```
>>> from math import sqrt  
>>> trier(l, a_comparer=lambda z:sqrt(z.real**2+z.imag**2))  
>>> l  
[(2+2j), (3+1j), (1+3j)]
```

I. Expressions Lambda

Conclusion

Avantages

- sont des fonctions anonymes très simples et concises
- sont issues de la programmation fonctionnelle
- permettent de généraliser des algorithmes
- permettent d'éviter de créer des fonctions peu utiles

Limitations

- Ne peut contenir qu'une seule expression.
- Moins adaptée pour des opérations complexes.

Fonctions d'ordre supérieur

- Définir et comprendre les fonctions d'ordre supérieur.
- Explorer l'utilisation des fonctions intégrées (`map()`, `filter()`, et `reduce()`) avec lambdas.
- Mettre en œuvre des techniques avancées avec des fonctions d'ordre supérieur

Fonctions d'Ordre Supérieur

Définition :

Fonctions qui :

- acceptent d'autres fonctions en argument.
- peuvent retourner une fonction.

Exemples de fonction d'ordre supérieur :

- `map()`
- `filter()`
- `reduce()`

II. Fonctions d'ordre supérieur

La fonction `map()` :

Définition :

Applique une fonction à chaque élément d'un **iterable**.

Syntaxe :

```
map(function, iterable)
```

Où :

- `iterable` : tout objet capable de renvoyer ses éléments un par un.

Exemple :

```
nombres = [1, 2, 3, 4]
carres = list(map(lambda x: x ** 2, nombres))
print(carres)  # [1, 4, 9, 16]
```

II. Fonctions d'ordre supérieur

La fonction `Filter()`

Définition :

Filtre les éléments selon une condition.

Syntaxe :

```
filter(function, iterable)
```

Exemple :

```
nombres = [1, 2, 3, 4, 5]
pairs = list(filter(lambda x: x % 2 == 0, nombres))
print(pairs)  # [2, 4]
```

II. Fonctions d'ordre supérieur

Fonction reduce

Définition

Réduit un **iterable** en un seul résultat par une opération cumulative. Nécessite **functools**.

Syntaxe

```
from functools import reduce  
reduce(function, iterable[, initializer])
```

- **function** : Une fonction à deux arguments qui sera appliquée de manière cumulative à l'itérable.
- **iterable** : Un itérable (comme une liste ou un tuple) sur lequel la fonction sera appliquée.
- **initializer (optionnel)** : Une valeur initiale à utiliser comme premier argument lors de la première application de la fonction. Si ce paramètre est fourni, il est ajouté au début de l'itérable.

II. Fonctions d'ordre supérieur

Fonction reduce

Exemple : Calcul de la somme

```
from functools import reduce
nombres = [1, 2, 3]
somme = reduce(lambda x, y: x + y, nombres)
print(somme) # 6
somme = reduce(lambda x, y: x + y, nombres, 10)
print(somme) # Affiche : 16
```


II. Fonctions d'ordre supérieur

Fonctions Cache

Définition :

Permettent de mémoriser les résultats de fonctions.

- `lru_cache` (maxsize limité) et `cache`(illimité).

Exemple avec `lru_cache` :

```
@lru_cache(maxsize=3)
def calcul_lent(x):
    return x ** 2
```

Exemple avec `cache` :

```
@cache
def calcul_lent(x):
    return x ** 2
```

II. Fonctions d'ordre supérieur

Exercices

Ex1 : Filtrer les nombres impairs et calculer le produit de leurs cubes.

Ex2 : Trier une liste par la somme des chiffres.

II. Fonctions d'ordre supérieur

Solution

Ex1 :

```
from functools import cache, reduce

@cache
def cube(x):
    return x ** 3

nombres = [1, 2, 3, 4, 5]
produit_cubes = reduce(
    lambda x, y: x * y,
    map(cube, filter(lambda x: x % 2 != 0, nombres))
)
print(produit_cubes)  # 3375
```

II. Fonctions d'ordre supérieur

Solution

Ex2:

```
from functools import lru_cache

@lru_cache(maxsize=10)
def somme_chiffres(x):
    return sum(int(chiffre) for chiffre in str(x))

nombres = [123, 45, 6, 789]
nombres_tries = sorted(nombres, key=somme_chiffres)
print(nombres_tries)  # [6, 45, 123, 789]
```